

强化学习算法公式与伪代码复习

重点：考试更可能考算法思想、Bellman 方程、更新公式、伪代码填空、算法对比、情景分析。
这份文档按“会写公式、会补伪代码、会解释为什么”整理。

1. 强化学习基本符号

符号	含义
S	状态空间
A	动作空间
s	当前状态
a	当前动作
r	即时奖励
s'	下一状态
a'	下一动作
$\pi(a s)$	策略，在状态 s 下选择动作 a 的概率
$V(s)$	状态价值函数
$Q(s,a)$	动作价值函数
γ	折扣因子
α	学习率
ϵ	epsilon-greedy 探索率

回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

智能体目标：

$$\max E[G_t]$$

2. Bellman 方程

2.1 状态价值函数

给定策略 π ：

$$V_{\pi}(s) = E_{\pi}[G_t \mid S_t=s]$$

Bellman 期望方程：

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_{\pi}(s')]$$

2.2 动作价值函数

$$Q_{\pi}(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q_{\pi}(s',a')]$$

2.3 最优价值函数

最优状态价值：

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')]$$

最优动作价值：

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \max_{a'} Q^*(s',a')]$$

最优策略：

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$$

3. 价值迭代算法

3.1 核心思想

价值迭代直接用 Bellman 最优方程更新 $V(s)$ ：

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$$

收敛后，根据价值函数导出策略：

$$\pi(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V(s')]$$

3.2 价值迭代伪代码

输入：状态空间 S，动作空间 A，转移概率 P，奖励函数 R，折扣率 γ ，阈值 θ
输出：最优策略 π

初始化：
 对所有状态 s，初始化 $V(s) = 0$

repeat：
 $\Delta = 0$
 for 每个状态 s in S：
 $v = V(s)$
 $V(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V(s')]$
 $\Delta = \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

for 每个状态 s in S：
 $\pi(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V(s')]$

返回 π

3.3 考试答题要点

- 价值迭代不显式做完整策略评估。
- 每次更新都直接对动作取最大值。
- 先求最优价值函数，再从价值函数导出最优策略。
- 适用于模型已知的 MDP。

4. 策略迭代算法

4.1 核心思想

策略迭代由两步交替组成：

策略评估：固定 π ，求 V_{π}
策略提升：根据 V_{π} 变贪婪，得到更好的 π

4.2 策略评估公式

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_{\pi}(s')]$$

迭代形式：

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$$

4.3 策略提升公式

$$\pi_{new}(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_{\pi}(s')]$$

4.4 策略迭代伪代码

输入：状态空间 S，动作空间 A，转移概率 P，奖励函数 R，折扣率 gamma，阈值 theta
输出：最优策略 pi

```
初始化：
    任意初始化策略 pi
    对所有状态 s，初始化 V(s) = 0

repeat:
    # 1. 策略评估
    repeat:
        Delta = 0
        for 每个状态 s in S:
            v = V(s)
            V(s) = sum_a pi(a|s) sum_{s'} P(s'|s,a) [R(s,a,s') + gamma V(s')]
            Delta = max(Delta, |v - V(s)|)
        until Delta < theta

    policy_stable = true

    # 2. 策略提升
    for 每个状态 s in S:
        old_action = pi(s)
        pi(s) = argmax_a sum_{s'} P(s'|s,a) [R(s,a,s') + gamma V(s')]
        if old_action != pi(s):
            policy_stable = false

until policy_stable == true

返回 pi
```

4.5 策略迭代 vs 价值迭代

对比	策略迭代	价值迭代	
核心过程	策略评估 + 策略提升	直接 Bellman 最优更新	
是否显式维护策略	是	通常最后才导出策略	
更新公式	对动作按 $\pi(a$	$s)$ 加权	对动作取 $\max$
收敛判断	策略不再变化	价值变化小于阈值	
模型要求	需要模型	需要模型	

一句话区别：

策略迭代：先评价当前策略有多好，再改策略。
价值迭代：每次都假设自己选最优动作，直接逼近最优价值。

5. MC、TD、SARSA、Q-learning

5.1 MC 蒙特卡洛

MC 用完整回合的真实回报更新：

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

或：

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha [G_t - Q(S_t,A_t)]$$

特点：

- 不需要环境模型。
- 必须等一个 episode 结束。
- 不使用 bootstrap。
- 方差较大。

5.2 TD(0)

TD 用一步奖励加下一状态价值更新：

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD 误差：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

特点：

- 不需要模型。
- 不必等回合结束。
- 使用 bootstrap。

5.3 SARSA

SARSA 是同策略 on-policy TD 控制算法。

名字来自一组转移：

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$$

更新公式：

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)]$$

SARSA 伪代码

输入：状态空间 S，动作空间 A，折扣率 gamma，学习率 alpha，探索率 epsilon
输出：策略 pi

对所有 s,a，随机初始化 Q(s,a)
根据 Q 构造 epsilon-greedy 策略 pi

```
repeat:
    初始化初始状态 s
    根据 epsilon-greedy 策略选择动作 a

    repeat:
        执行动作 a，得到奖励 r 和新状态 s'
        在状态 s'，根据 epsilon-greedy 策略选择动作 a'

        Q(s,a) <- Q(s,a) + alpha [r + gamma Q(s',a') - Q(s,a)]

        根据新的 Q 更新 epsilon-greedy 策略 pi

        s <- s'
        a <- a'

    until s 为终止状态

until Q(s,a) 收敛

返回 pi
```

图片中 SARSA 伪代码空白处

如果题目让补空，一般填：

Q(s,a) <- Q(s,a) + alpha [r + gamma Q(s',a') - Q(s,a)]

以及：

pi <- 根据 Q 更新得到的 epsilon-greedy 策略

或者写成：

pi(s) = argmax_a Q(s,a) # 若要求贪婪策略
pi = epsilon-greedy(Q) # 若要求探索策略

注意：SARSA 必须用实际选出来的 a'，不能写 max_a Q(s',a)。

5.4 Q-learning

Q-learning 是异策略 off-policy TD 控制算法。

更新公式：

Q(S_t,A_t) <- Q(S_t,A_t)
 + alpha [R_{t+1} + gamma max_a Q(S_{t+1},a) - Q(S_t,A_t)]

Q-learning 伪代码

输入：状态空间 S，动作空间 A，折扣率 gamma，学习率 alpha，探索率 epsilon
输出：策略 pi

对所有 s,a，随机初始化 Q(s,a)

```
repeat:
    初始化初始状态 s

    repeat:
        根据 epsilon-greedy 策略从 Q 中选择动作 a
        执行动作 a，得到奖励 r 和新状态 s'

        Q(s,a) <- Q(s,a) + alpha [r + gamma max_a' Q(s',a') - Q(s,a)]

        s <- s'

    until s 为终止状态

until Q(s,a) 收敛

对所有状态 s:
    pi(s) = argmax_a Q(s,a)

返回 pi
```

5.5 SARSA vs Q-learning

对比	SARSA	Q-learning
类型	On-policy	Off-policy
更新目标	r + gamma Q(s',a')	r + gamma max_a Q(s',a)
下一动作	用实际选择的 a'	不关心实际下一动作
是否考虑探索风险	考虑	不直接考虑
悬崖漫步	更保守	更接近最短路径但风险更高

6. Dyna-Q

6.1 核心思想

Dyna-Q = Q-learning + 模型学习 + 规划。

真实交互得到：

(s, a, r, s')

先做 Q-learning：

Q(s,a) <- Q(s,a) + alpha [r + gamma max_a' Q(s',a') - Q(s,a)]

再存模型：

Model(s,a) <- r, s'

然后从模型中随机抽过去的经验做 planning：

Q(s,a) <- Q(s,a) + alpha [r + gamma max_a' Q(s',a') - Q(s,a)]

6.2 Dyna-Q 伪代码

输入：状态空间 S，动作空间 A，学习率 alpha，折扣率 gamma，探索率 epsilon，规划步数 n
输出：策略 pi

初始化 Q(s,a)
初始化模型 Model

```
repeat:
    初始化状态 s

    repeat:
        根据 epsilon-greedy 从 Q 中选择动作 a
        执行动作 a，得到 r 和 s'

        Q(s,a) <- Q(s,a) + alpha [r + gamma max_a' Q(s',a') - Q(s,a)]

        Model(s,a) <- r, s'

    repeat n 次:
        从已访问过的状态动作对中随机选 (s_m, a_m)
        从 Model 中得到 r_m, s_m'
        Q(s_m,a_m) <- Q(s_m,a_m)
            + alpha [r_m + gamma max_a' Q(s_m',a') - Q(s_m,a_m)]

    s <- s'

until s 为终止状态

until Q 收敛

pi(s) = argmax_a Q(s,a)
返回 pi
```

6.3 常考点

- n=0 时，Dyna-Q 退化为普通 Q-learning。
- n 越大，模型规划次数越多，通常学习更快，但计算量更大。
- Dyna-Q 是 model-based，因为它学习并使用了 Model(s,a)。

7. DQN

DQN 是用神经网络近似 Q(s,a) 的 Q-learning。

目标值：

y = r + gamma max_a' Q_target(s',a')

损失函数：

L(theta) = [y - Q(s,a;theta)]^2

两个关键技巧：

经验回放 Replay Buffer：随机采样历史经验，打破样本相关性。
目标网络 Target Network：固定一段时间的目标 Q，稳定训练。

8. REINFORCE、Actor-Critic、TRPO、PPO

8.1 REINFORCE

REINFORCE 是蒙特卡洛策略梯度算法。

目标：

J(theta) = E_pi[G_t]

策略梯度：

grad J(theta) = E_pi [G_t grad log pi_theta(A_t|S_t)]

参数更新：

theta <- theta + alpha G_t grad log pi_theta(A_t|S_t)

特点：

- 直接优化策略。
- On-policy。
- 需要完整轨迹。
- 方差较大。

8.2 Actor-Critic

Actor-Critic 包含两个部分：

Actor：策略 pi_theta(a|s)，负责选择动作。
Critic：价值函数 V_w(s) 或 Q_w(s,a)，负责评价动作好坏。

TD 误差：

delta = r + gamma V_w(s') - V_w(s)

Critic 更新：

w <- w + beta delta grad_w V_w(s)

Actor 更新：

theta <- theta + alpha delta grad_theta log pi_theta(a|s)

一句话：

Actor 负责做决策，Critic 负责打分，Actor 根据 Critic 的反馈改进策略。

8.3 TRPO

TRPO 用 KL 散度限制策略更新幅度。

目标：

maximize E[pi_theta(a|s) / pi_old(a|s) * A_old(s,a)]

约束：

E[KL(pi_old(.|s), pi_theta(.|s))] <= delta

特点：

- 更新稳定。
- 理论性强。
- 实现复杂。

8.4 PPO

PPO 是 TRPO 的简化版本，常考 PPO-Clip。
概率比：

$$r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{old}(a_t|s_t)$$

Clip 目标：

$$L_{clip}(\theta) = E_t [\min(r_t(\theta) A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t)]$$

直觉：

如果新策略和旧策略差太远，就截断 ratio ，防止更新过猛。

常考点：

- PPO 是 on-policy 策略梯度算法。
- PPO 通常使用 Actor-Critic 框架。
- A_t 是优势函数，表示动作比平均水平好多少。
- clip 限制新旧策略概率比。

9. 迷宫奖励题：走出迷宫 +1，否则 0，gamma = 1 能否学会？

题型示例：

一个智能体通过强化学习学习走迷宫。如果走出迷宫，奖励为 +1；否则奖励为 0。智能体目标是最大化期望回报。当折扣率 $\gamma = 1$ 时，智能体是否能学会走迷宫技巧？如何改进？

9.1 标准分析

如果任务是有限回合 episodic，且智能体最终能到达出口，那么理论上可以学会。因为成功走出迷宫会得到回报：

$$G_t = 1$$

没走出去则：

$$G_t = 0$$

策略会倾向于提高走出迷宫的概率。

但是当 $\gamma = 1$ 且中间奖励全为 0 时，会有几个问题：

1. 奖励非常稀疏，只有走出迷宫才有信号。
2. 所有能走出去的路径回报都一样，都是 1，无法区分“快走出去”和“慢走出去”。
3. 如果存在循环或无限步， $\gamma = 1$ 可能导致回报不收敛，智能体可能长时间无有效学习信号。
4. 探索不足时，智能体可能很久都碰不到出口，因此学不会。

所以更准确的答案是：

在有限回合并且有足够探索的情况下，可能学会走出迷宫；
但 $\gamma=1$ 且奖励稀疏，会导致学习效率低，且不能鼓励最短路径。

9.2 如何改进

常见改进方法：

1. 设置 $\gamma < 1$ 。

$\gamma < 1$ 时，越早得到 +1，回报越大。

例如：

走 3 步到出口： $G = \gamma^3 * 1$
走 10 步到出口： $G = \gamma^{10} * 1$

当 $0 < \gamma < 1$ ：

$$\gamma^3 > \gamma^{10}$$

因此智能体会偏向更短路径。

1. 加入每步惩罚。

例如每走一步奖励 -0.01 ，走出迷宫奖励 +1：

$r = -0.01$ ，每步
 $r = +1$ ，出口

这样路径越长，累计惩罚越多，可以鼓励智能体尽快走出迷宫。

1. 奖励塑形 reward shaping。

例如离出口更近给小的正奖励，远离出口给小的负奖励。

1. 增强探索。

例如使用 epsilon-greedy，并让 epsilon 逐渐衰减。

9.3 可直接背诵答案

不一定能有效学会。若任务是有限回合且探索充分， $\gamma=1$ 时智能体理论上可以通过终点奖励 +1 学会走出迷宫；但由于中间奖励均为 0，奖励非常稀疏，而且所有成功路径回报都相同，无法鼓励智能体选择更短路径。若存在循环或无限步， $\gamma=1$ 还可能致回报不收敛。

改进方法包括：将 γ 设置为小于 1，使越早到达终点回报越大；给每一步加入小的负奖励，例如 -0.01 ；进行奖励塑形；使用 epsilon-greedy 等方法增强探索。

10. 高频简答题

10.1 为什么强化学习需要探索和利用？

答：

利用是选择当前认为最优的动作，以获得较高回报；探索是尝试其他动作，以发现可能更优的策略。如果只利用，容易陷入局部最优；如果只探索，学习效率低。因此需要在探索和利用之间权衡。

10.2 epsilon-greedy 是什么？

答：

以 $1-\epsilon$ 的概率选择当前 Q 值最大的动作，以 ϵ 的概率随机选择动作。

公式：

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \epsilon / |A|, & \text{如果 } a = \operatorname{argmax}_a Q(s,a) \\ \epsilon / |A|, & \text{否则} \end{cases}$$

10.3 On-policy 和 Off-policy 区别

答：

On-policy 使用当前实际执行的策略产生的数据来评估和改进同一个策略，例如 SARSA。
Off-policy 使用一个行为策略产生数据，但学习另一个目标策略，例如 Q-learning。

10.4 MC 和 TD 区别

对比	MC	TD
是否等 episode 结束	是	否
是否 bootstrap	否	是
目标	实际回报 G_t	$r + \gamma V(s')$
方差	较大	较小
偏差	无 bootstrap 偏差	有 bootstrap 偏差

10.5 DP 和 TD 区别

对比	DP	TD
是否需要模型	需要	不需要
数据来源	已知转移概率	与环境交互采样
更新方式	遍历状态动作	根据经验样本
代表算法	策略迭代、价值迭代	TD、SARSA、Q-learning

11. 公式速背

Bellman 期望方程：

```
V_pi(s) = sum_a pi(a|s) sum_s' P(s'|s,a) [r + gamma V_pi(s')]
```

Bellman 最优方程：

```
V*(s) = max_a sum_s' P(s'|s,a) [r + gamma V*(s')]
```

MC：

```
V(S_t) <- V(S_t) + alpha [G_t - V(S_t)]
```

TD(0)：

```
V(S_t) <- V(S_t) + alpha [R_{t+1} + gamma V(S_{t+1}) - V(S_t)]
```

SARSA：

```
Q(S,A) <- Q(S,A) + alpha [R + gamma Q(S',A') - Q(S,A)]
```

Q-learning：

```
Q(S,A) <- Q(S,A) + alpha [R + gamma max_a Q(S',a) - Q(S,A)]
```

Dyna-Q：

```
真实更新: Q-learning
模型保存: Model(S,A) <- R,S'
规划更新: 从 Model 抽样后继续 Q-learning
```

REINFORCE：

```
theta <- theta + alpha G_t grad log pi_theta(A_t|S_t)
```

Actor-Critic：

```
delta = r + gamma V(s') - V(s)
theta <- theta + alpha delta grad log pi_theta(a|s)
```

PPO：

```
r_t(theta) = pi_theta(a_t|s_t) / pi_old(a_t|s_t)
L_clip = E[min(r_t A_t, clip(r_t,1-epsilon,1+epsilon) A_t)]
```

12. 考场判断口诀

```
看见 P(s'|s,a): 动态规划 DP。
看见完整回合 G_t: MC 或 REINFORCE。
看见 r + gamma V(s'): TD。
看见 Q(s',a'): SARSA。
看见 max Q(s',a): Q-learning。
看见经验回放和目标网络: DQN。
看见 Model(s,a): Dyna-Q。
看见 grad log pi: 策略梯度。
看见 Actor 和 Critic: Actor-Critic。
看见 KL 约束: TRPO。
看见 ratio 和 clip: PPO。
```

可以。下面我按你老师考纲里的算法，给你出一组类似刚才迷宫题的情景论述题，每题都偏考试风格，并附上参考答案要点。你可以先只看题目，自己写一版，再对照采分点。

强化学习考纲模拟题

覆盖：

```
Model: DP, VI, PI, Dyna
Value: MC, TD, SARSA, Q-learning, DQN
Policy: REINFORCE, Actor-Critic, TRPO, PPO
```

1. DP / VI / PI：已知模型时如何求最优策略

题目 1，20 分

在一个 (4 times 4) 的 Grid World 中，智能体可以上下左右移动。环境的状态转移概率 (p(s'|s,a)) 和奖励函数 (r(s,a)) 都完全已知。到达终点奖励为 (+1)，每走一步奖励为 (-0.01)，撞墙奖励为 (-0.1)，折扣因子 (gamma = 0.9)。

- 请问：
1. 这个问题是否适合使用动态规划 DP？为什么？
 2. 分别说明策略迭代 PI 和价值迭代 VI 的基本步骤。
 3. 如果环境模型未知，DP 是否还能直接使用？应该改用什么方法？

参考要点

这题考 **model-based DP**。

DP 适用，因为：

```
状态空间有限
动作空间有限
p(s'|s,a) 已知
r(s,a) 已知
```

策略迭代 PI：

```
初始化策略 π
重复：
    策略评估: 计算 Vπ(s)
    策略改进: 根据 Vπ(s) 贪心改进策略
直到策略稳定
```

策略评估公式：

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) [r(s,a) + \gamma V_{\pi}(s')]$$

策略改进：

$$\pi'(s) = \arg \max_a \sum_{s'} p(s'|s,a) [r(s,a) + \gamma V_{\pi}(s')]$$

价值迭代 VI：

$$V_{k+1}(s) = \max_a \sum_{s'} p(s'|s,a) [r(s,a) + \gamma V_k(s')]$$

如果模型未知，不能直接用 DP，可以用：

- MC
- TD
- SARSA
- Q-learning
- DQN

一句话答案：

DP 适合模型已知的小规模 MDP；PI 是评估策略再改进策略，VI 是直接 Bellman 最优方程更新价值。

2. PI 和 VI 的区别

题目 2，15 分
某同学说：“策略迭代和价值迭代都能求最优策略，所以它们没有本质区别。”请判断这句话是否正确，并说明 PI 和 VI 的区别。

参考要点
这句话不完全正确。
共同点：

- 都属于动态规划
- 都要求已知环境模型
- 都基于 Bellman 方程
- 都可以求最优策略

对比	PI	VI
全称	Policy Iteration	Value Iteration
核心	策略评估 + 策略改进	直接更新最优价值
每轮计算量	较大	较小
收敛轮数	通常较少	通常较多
是否完整评估策略	是	不一定

PI 的结构：

- 先算当前策略到底多好
- 再根据价值改进策略

VI 的结构：

- 每次直接向最优价值靠近

考试金句：

PI 每轮有明确的策略评估和策略改进；VI 把策略评估截断为一步，直接使用 Bellman 最优方程迭代。

3. Dyna-Q：真实经验 + 模拟经验

题目 3，20 分
一个机器人在迷宫中学习走到出口。真实环境交互代价很高，每走一步都需要消耗电量。已知机器人可以通过过去的经验学习一个环境模型，并利用这个模型进行模拟训练。
请问：

- 应该使用什么类型的强化学习算法？
- 说明 Dyna-Q 的基本流程。
- Dyna-Q 相比普通 Q-learning 有什么优点？
- 如果学习到的模型不准确，会产生什么问题？

参考要点
适合使用：

- Dyna-Q

Dyna-Q 结合了：

- model-free learning + model-based planning

基本流程：

- 真实环境中执行动作 a
- 得到真实经验 (s, a, r, s')
- 用真实经验更新 $Q(s,a)$
- 用真实经验更新环境模型 $Model(s,a) = (r,s')$
- 从模型中采样模拟经验
- 用模拟经验继续更新 $Q(s,a)$

Q-learning 更新公式：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s,a') - Q(s,a)]$$

优点：

- 提高样本利用率
- 减少真实交互次数
- 学习速度通常比普通 Q-learning 更快
- 可以利用模拟经验进行 planning

问题：

- 模型错误会导致错误规划
- 模拟经验可能误导 Q 值
- 环境变化后旧模型可能过时

一句话答案：

Dyna-Q 是一边在真实世界学习，一边在脑内小迷宫里模拟练习。

4. MC：完整回合后更新

题目 4，20 分

智能体玩一个回合制游戏。只有游戏结束时才会得到奖励：胜利 (+1)，失败 (-1)，中间步骤奖励均为 0。智能体希望估计每个状态的价值。请问：

- 1. 是否可以使用蒙特卡洛 MC 方法？
- 2. MC 方法如何更新状态价值函数？
- 3. MC 方法和 TD 方法相比有什么优缺点？
- 4. 如果游戏回合非常长，MC 会遇到什么问题？

参考要点

可以使用 MC，因为这是 episodic task，能等到一局结束后计算完整回报。
回报：

```
[
G_t = R_{t+1}+\gamma R_{t+2}+\gamma^2R_{t+3}+\cdots
]
```

MC 更新：

```
[
V(S_t)\leftarrow V(S_t)+\alpha[G_t - V(S_t)]
]
```

MC 特点：

不需要环境模型
必须等 episode 结束
使用真实完整回报
不使用 bootstrap

MC vs TD：

对比	MC	TD
更新时间	回合结束后	每走一步
是否需要完整 episode	需要	不需要
是否 bootstrap	否	是
方差	较大	较小
偏差	较小	有估计偏差

长回合的问题：

更新太慢
方差大
回报延迟严重
学习效率低

一句话答案：

MC 像打完整局后复盘，结果真实但反馈慢。

5. TD：一步更新

题目 5，15 分

某智能体在状态 (s) 的当前价值估计为：

```
[
V(s)=0.3
]
```

执行动作后得到奖励：

```
[
r=0.2
]
```

转移到下一个状态 (s')，且：

```
[
V(s')=0.8
]
```

设学习率 ($\alpha=0.1$)，折扣因子 ($\gamma=0.9$)。请使用 TD(0) 更新 (V(s))。

参考要点

TD(0) 更新公式：

```
[
V(s)\leftarrow V(s)+\alpha[r+\gamma V(s')-V(s)]
]
```

TD target：

```
[
r+\gamma V(s')=0.2+0.9\times 0.8=0.92
]
```

TD error：

```
[
0.92-0.3=0.62
]
```

更新后：

```
[
V(s)=0.3+0.1\times 0.62=0.362
]
```

答案：

```
[
V(s)=0.362
]
```

考试提醒：

TD 的核心是用 ($r+\gamma V(s')$) 作为当前状态价值的估计目标。

6. SARSA 和 Q-learning：悬崖环境

题目 6，20 分

在 Cliff Walking 环境中，智能体从起点出发走向终点，悬崖区域奖励为 (-100)，普通移动奖励为 (-1)。智能体使用 (ϵ)-greedy 策略探索。

请问：

- 1. SARSA 和 Q-learning 分别属于 on-policy 还是 off-policy？
- 2. 写出它们的更新公式。
- 3. 在悬崖环境中，哪种算法通常更保守？为什么？

4. 两者最终学到的策略可能有什么不同？

参考要点

SARSA 是：

```
on-policy
```

Q-learning 是：

```
off-policy
```

SARSA 更新：

```
[
Q(S,A)\leftarrow Q(S,A)+\alpha[R+\gamma Q(S',A')-Q(S,A)]
]
```

Q-learning 更新：

```
[
Q(S,A)\leftarrow Q(S,A)+\alpha[R+\gamma \max_{a'}Q(S',a')-Q(S,A)]
]
```

区别：

对比	SARSA	Q-learning
类型	on-policy	off-policy
下一个动作	实际执行的 (A')	最大 Q 值对应动作
是否考虑探索风险	考虑	不充分考虑
风格	保守	激进

Cliff Walking 中：

```
SARSA 通常会远离悬崖
Q-learning 更可能贴近悬崖走最短路径
```

原因：

```
SARSA 更新时考虑了 (\epsilon)-greedy 探索带来的掉悬崖风险；Q-learning 假设下一步总能选择最优动作，因此更激进。
```

7. SARSA 和 Q-learning 数值计算题

题目 7，15 分

已知当前：

```
[
Q(s,a)=0.5
]
```

执行动作后得到：

```
[
r=0.2
]
```

设：

```
[
\gamma=0.9,\quad \alpha=0.1
]
```

在下一个状态 (s') 中，智能体实际选择的动作 (a') 满足：

```
[
Q(s',a')=0.4
]
```

而所有动作中最大 Q 值为：

```
[
\max_{a'}Q(s',a')=0.8
]
```

请分别计算 SARSA 和 Q-learning 更新后的 (Q(s,a))。

参考要点

SARSA target：

```
[
0.2+0.9\times 0.4=0.56
]
```

SARSA 更新：

```
[
Q(s,a)=0.5+0.1(0.56-0.5)=0.506
]
```

Q-learning target：

```
[
0.2+0.9\times 0.8=0.92
]
```

Q-learning 更新：

```
[
Q(s,a)=0.5+0.1(0.92-0.5)=0.542
]
```

答案：

```
SARSA: 0.506
Q-learning: 0.542
```

这题非常适合考，因为它把 on-policy 和 off-policy 的区别钉在公式上了。

8. DQN：为什么需要神经网络

题目 8，20 分

一个智能体玩 Atari 游戏，状态是游戏画面像素，动作是上下左右和开火。由于状态空间巨大，无法用表格记录每个状态动作对的 Q 值。

请问：

- 1. 为什么普通 Q-learning 不适合这个任务？
- 2. DQN 如何解决这个问题？
- 3. DQN 的目标值和损失函数是什么？
- 4. DQN 中经验回放和目标网络分别有什么作用？

参考要点

普通 Q-learning 不适合，因为：

```
状态空间巨大
无法建立 Q-table
相似状态之间无法泛化
像素输入维度太高
```

DQN 使用神经网络近似 Q 函数：

```
[
  Q_(theta(s,a))
]
```

输入状态 (s)，输出每个动作的 Q 值。

DQN target：

```
[
  y = r+gamma max_{a'}Q_{(theta^{~})}(s',a')
]
```

其中 $(\theta^{~})$ 是目标网络参数。

损失函数：

```
[
  L(theta) = [y-Q_(theta(s,a))]^2
]
```

经验回放作用：

```
存储历史经验 (s,a,r,s',done)
随机采样 batch 训练
打破样本相关性
提高样本利用率
```

目标网络作用：

```
稳定 target
避免当前网络自己追自己
降低训练震荡
```

一句话答案：

DQN 是用神经网络代替 Q 表，并通过经验回放和目标网络让训练不至于像喝了三杯浓缩咖啡一样乱跳。

9. DQN 数值计算题

题目 9，10 分

在 DQN 中，某条经验为：

```
[
(s,a,r,s')
]
```

其中：

```
r = 1
gamma = 0.99
max Q_target(s',a') = 4.0
当前网络 Q(s,a) = 3.2
```

假设 (s') 不是终止状态。请计算 DQN target 和平方损失。
如果 (s') 是终止状态，target 又是多少？

参考要点

非终止状态：

```
[
y = r+gamma max_{a'}Q_{(theta^{~})}(s',a')
]
```

```
[
y = 1+0.99*4.0=4.96
]
```

平方损失：

```
[
L=(4.96-3.2)^2=1.76^2=3.0976
]
```

如果 (s') 是终止状态：

```
[
y=r=1
]
```

答案：

```
非终止 target = 4.96
loss = 3.0976
终止状态 target = 1
```

10. REINFORCE：直接优化策略

题目 10，20 分

一个智能体使用随机策略 $(\pi_{\theta}(a|s))$ 玩游戏。游戏只有在结束时才给奖励，胜利 (+1)，失败 (-1)。智能体不学习 Q 表，而是直接调整策略参数 (θ) 。

请问：

- 这属于哪类强化学习方法？
- 写出 REINFORCE 的策略梯度更新形式。
- 为什么 REINFORCE 的方差较大？
- 如何降低方差？

参考要点

这是：

```
policy-based
policy gradient
REINFORCE
```

目标：

```
[
J(theta)=E{pi*(theta)[G_t]}
]
```

策略梯度：

```
[
nabla_theta J(theta)=E[nabla_theta log pi_theta(a_t|s_t)G_t]
]
```

参数更新：

```
[
theta -> theta+alpha nabla_theta log pi_theta(a_t|s_t)G_t
]
```

直觉：

如果某个动作带来高回报，提高以后选它的概率
如果某个动作带来低回报，降低以后选它的概率

方差大的原因：

使用完整回报 G_t
回报受整条轨迹随机性影响
奖励延迟
采样效率低

降低方差方法：

加入 $baseline$
使用优势函数 $advantage$
使用 Actor-Critic
对回报归一化
增加采样数量

带 $baseline$ 的形式：

```
[
\nabla_{\theta} J(\theta)=\mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)(G_t-b(s_t))]
]
常用 baseline:
[
b(s_t)=V(s_t)
]
---
```

11. Actor-Critic：演员和评论家

题目 11，20 分

某智能体使用一个网络输出动作概率，另一个网络估计状态价值 $V(s)$ 。在训练时，价值网络用 TD error 评价当前动作的好坏，然后指导策略网络更新。

请问：

1. 这是哪种算法框架？
2. Actor 和 Critic 分别负责什么？
3. 写出 TD error 的形式。
4. Actor-Critic 相比 REINFORCE 有什么优点？

参考要点

这是：

Actor-Critic

Actor：

学习策略 $\pi_{\theta}(a|s)$
负责选择动作

Critic：

学习价值函数 $V_w(s)$ 或 $Q_w(s,a)$
负责评价 Actor 的动作

TD error：

```
[
\delta_t = r_t+\gamma V_w(s_{t+1})-V_w(s_t)
]
```

Actor 更新方向：

```
[
\theta \leftarrow \theta+\alpha \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)\delta_t
]
```

Critic 通过最小化 TD error 更新：

```
[
L(w)=\delta_t^2
]
```

优点：

相比 REINFORCE 方差更低
不一定要等完整 episode 结束
可以在线更新
Critic 提供更稳定的学习信号

缺点也可以写：

Critic 估计不准会误导 Actor
训练可能不稳定

一句话答案：

Actor 负责“怎么做”，Critic 负责“做得怎么样”。

12. TRPO：为什么要限制策略更新

题目 12，20 分

在策略梯度算法中，某智能体每次更新策略参数时步长过大，导致新策略和旧策略差别很大，训练过程中性能突然下降，甚至崩溃。

请问：

1. 这种问题产生的原因是什么？
2. TRPO 如何解决这个问题？
3. TRPO 中 KL 散度约束的作用是什么？
4. TRPO 的优缺点是什么？

参考要点

问题原因：

策略更新过大
新旧策略差异过大
一次更新破坏原来较好的策略
策略梯度估计有噪声

TRPO：

Trust Region Policy Optimization
信任域策略优化

核心思想：

每次策略更新不能离旧策略太远。

TRPO 优化形式大致是：

[

```
\max_{\theta} L(\theta)
]
subject to:
[
D_{KL}(\pi_{\theta_{old}}||\pi_{\theta}) \leq \delta
]
```

KL 散度作用：

衡量新旧策略差异

限制策略变化幅度

提高训练稳定性

防止性能突然崩溃

优点：

训练稳定

理论保证较强

适合处理策略更新过大问题

缺点：

实现复杂

计算量大

需要二阶近似或共轭梯度

工程上不如 PPO 简单

一句话答案：

TRPO 给策略更新划了一个安全区，别让策略一步蹦到悬崖外。

13. PPO：Clip 的作用

题目 13，20 分

PPO 是 TRPO 的一种简化实用算法。它不用复杂的 KL 约束，而是通过 clipped surrogate objective 限制策略更新幅度。

- 请问：
- PPO 中概率比 $(r_t(\theta))$ 的定义是什么？
 - PPO-Clip 的目标函数是什么？
 - clip 操作的作用是什么？
 - PPO 相比 TRPO 有什么优点？

参考要点

概率比：

```
[
r_t(\theta)=\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}
]
```

PPO-Clip 目标：

```
[
L^{CLIP}(\theta)=\mathbb{E}_t[\min(r_t(\theta)A_t,\text{clip}(r_t(\theta),1-\epsilon,1+\epsilon)A_t)]
]
```

clip 作用：

限制新旧策略差异

防止策略更新过大

提高训练稳定性

避免性能突然崩溃

当 $(A_t>0)$ 时：

说明这个动作比平均水平好

希望提高该动作概率

但不能提高得太夸张

当 $(A_t<0)$ 时：

说明这个动作不好

希望降低该动作概率

但不能降低得太夸张

PPO 优点：

比 TRPO 简单

不需要复杂二阶优化

训练稳定

实现方便

实际效果好

一句话答案：

PPO 用 clip 给策略更新装了限速器。

14. PPO 数值计算题

题目 14，10 分

设 PPO 中：

$\epsilon = 0.2$

$A_t = 2$

$r_t(\theta) = 1.5$

请计算：

```
[
r_tA_t
]
```

和 clipped 项：

```
[
\text{clip}(r_t,1-\epsilon,1+\epsilon)A_t
]
```

最终 PPO 目标中取哪一个？

参考要点

clip 范围：

```
[
[1-\epsilon,1+\epsilon]=[0.8,1.2]
]
```

原始项：

```
[
```

```
r_tA_t=1.5\times 2=3
]
clip 后：
[
\text{clip}{1.5,0.8,1.2}=1.2
]
[
1.2\times 2=2.4
]
PPO 取：
[
\min(3,2.4)=2.4
]
```

答案：

```
原始项 = 3
clip 项 = 2.4
最终取 2.4
```

解释：

因为 (A_t>0)，这个动作是好动作，但 PPO 不允许策略把它的概率提高得过猛。

15. 综合分类题：选算法

题目 15，20 分

请根据下列任务选择合适算法，并说明理由。

1. 已知完整环境模型，状态动作空间较小，希望求最优策略。
2. 环境模型未知，但可以完整玩完一局游戏后得到胜负奖励。
3. 环境模型未知，希望每走一步就更新价值。
4. 状态是图像，动作是离散按钮，Q 表无法存储。
5. 动作空间连续，希望直接学习随机策略。
6. 策略梯度训练不稳定，希望限制新旧策略差异。
7. 希望用 TRPO 的思想，但实现更简单。

参考答案

1. 用 DP / PI / VI。

因为模型已知，状态动作空间较小。

1. 用 MC / REINFORCE。

如果估计价值，用 MC；如果直接优化策略，用 REINFORCE。

1. 用 TD / SARSA / Q-learning。

TD 可以一步更新。

1. 用 DQN。

因为 DQN 用神经网络近似 Q 函数。

1. 用 Policy Gradient / Actor-Critic / PPO。

连续动作更适合直接学习策略。

1. 用 TRPO。

通过 KL 约束限制策略更新幅度。

1. 用 PPO。

PPO 用 clip 近似限制策略更新，更简单实用。

最像考试会问的 8 个“压轴对比题”

这些建议重点背。

对比题 1：MC 和 TD

题目

MC 和 TD 都可以在未知模型下学习价值函数。请比较二者区别。

答题关键词

```
MC 等 episode 结束
TD 一步更新
MC 不 bootstrap
TD bootstrap
MC 方差大
TD 有偏差但方差较小
```

对比题 2：SARSA 和 Q-learning

题目

比较 SARSA 和 Q-learning，并说明为什么在 Cliff Walking 中 SARSA 更保守。

答题关键词

```
SARSA: on-policy, R + γQ(S',A')
Q-learning: off-policy, R + γ max Q(S',a')
SARSA 考虑实际探索风险
Q-learning 假设下一步最优
```

对比题 3：Q-learning 和 DQN

题目

DQN 和 Q-learning 有什么关系？为什么 DQN 需要经验回放和目标网络？

答题关键词

```
DQN = Q-learning + neural network
Q 表无法处理高维状态
经验回放打破相关性
目标网络稳定 TD target
```

对比题 4：REINFORCE 和 Actor-Critic

题目

比较 REINFORCE 和 Actor-Critic。

答题关键词

REINFORCE 直接用完整回报 G 更新策略
方差大
Actor-Critic 用 Critic 估计价值
用 TD error 或 advantage 更新 Actor
方差更低

对比题 5：DP 和 Dyna-Q

题目

DP 和 Dyna-Q 都利用模型，它们有什么区别？

答题关键词

DP 要求模型预先已知
Dyna-Q 从真实经验中学习模型
DP 直接规划
Dyna-Q = 真实学习 + 模拟规划

对比题 6：TRPO 和 PPO

题目

TRPO 和 PPO 都用于稳定策略优化，请比较二者。

答题关键词

TRPO 用 KL 约束
PPO 用 clip
TRPO 理论性强但复杂
PPO 简单稳定，工程常用

对比题 7：Value-based 和 Policy-based

题目

比较基于价值的方法和基于策略的方法。

答题关键词

Value-based 学 Q 或 V，再根据价值选动作
Policy-based 直接学习 $\pi(a|s)$
Value-based 适合离散动作
Policy-based 可处理连续动作和随机策略

对比题 8：On-policy 和 Off-policy

题目

什么是 on-policy 和 off-policy？举例说明。

答题关键词

on-policy：行为策略和目标策略相同
off-policy：行为策略和目标策略不同

SARSA 是 on-policy
Q-learning 是 off-policy
REINFORCE / PPO 通常是 on-policy
DQN 通常是 off-policy

一套背诵版总表

算法	最核心考点
DP	已知模型，用 Bellman 方程规划
PI	策略评估 + 策略改进
VI	Bellman 最优方程直接迭代
Dyna-Q	真实经验 + 学到的模型模拟
MC	完整 episode 后用真实回报更新
TD	一步更新，bootstrap
SARSA	on-policy，用实际 (A')
Q-learning	off-policy，用 $(\max Q)$
DQN	神经网络版 Q-learning
REINFORCE	蒙特卡洛策略梯度，方差大
Actor-Critic	Actor 学策略，Critic 学价值
TRPO	KL 约束，限制策略更新
PPO	Clip 概率比，简单稳定

这几个都属于 **Policy-based 方法**，也就是：

不再主要学习“每个状态/动作值多少钱”，而是直接学习一个策略 $\pi_i(\theta(a|s))$ ，让智能体知道在状态 (s) 下应该以多大概率选择动作 (a)。

先给你一张总览表：

算法	核心一句话	是否需要环境模型	常见采样方式
REINFORCE	用一整条 episode 的回报更新策略	不需要	on-policy
Actor-Critic	Actor 负责选动作，Critic 负责打分	不需要	常见 on-policy，也有 off-policy
TRPO	限制策略每次别更新太猛，用 KL 约束	不需要	on-policy
PPO	TRPO 的简化实用版，用 clip 限制更新	不需要	on-policy

1. Policy-based 方法在干什么？

以前你学的很多算法是 **Value-based**：

Q-learning：学习 $Q(s,a)$
SARSA：学习 $Q(s,a)$
DQN：用神经网络近似 $Q(s,a)$

它们的思路是：

先判断每个动作值多少钱，再选择价值最大的动作。

Policy-based 不一样，它直接学习策略：

$$\pi_{\theta}(a|s)$$

意思是：

在状态 s 下，选择动作 a 的概率是多少。

比如在一个状态下：

向左：0.1

向右：0.7

向上：0.1

向下：0.1

那策略就更倾向于向右。
Policy 方法的核心目标是最大化期望回报：

$$J(\theta)=\mathbb{E}[\pi^{\theta}[G_t]]$$

也就是调整参数 (θ) ，让策略拿到的总奖励越来越大。

2. REINFORCE：蒙特卡洛策略梯度

REINFORCE 是最经典、最基础的 **Policy Gradient** 算法。
它的思想是：

如果某个动作最后带来了高回报，就提高以后选这个动作的概率；如果回报差，就降低它的概率。

它像赛后复盘：

这一局我在 S_1 选了 A_1 ，最后赢了
说明 A_1 可能不错，以后在 S_1 更倾向选 A_1

这一局我在 S_2 选了 A_2 ，最后输了
说明 A_2 可能不好，以后少选 A_2

核心更新方向：
$$\nabla_{\theta} J(\theta)=\mathbb{E}[\left[G_t\nabla_{\theta}\log \pi_{\theta}(A_t|S_t)\right]]$$

这个公式不用怕，考试理解成：

回报 G_t 越大：
越增加当时动作的概率

回报 G_t 越小：
减少当时动作的概率

REINFORCE 的流程：

1. 用当前策略 π_{θ} 采样一整条 $episode$

2. $episode$ 结束后，计算每个时刻的回报 G_t

3. 对每个状态动作对 (S_t, A_t) ：
用 G_t 调整策略参数 θ

4. 重复很多轮

它和 MC 很像，因为它也必须等一整局结束。
所以：

$$REINFORCE = \text{蒙特卡洛方法} + \text{策略梯度}$$

优点：

不需要环境模型
可以直接优化随机策略
适合连续动作空间

缺点：

必须等 $episode$ 结束
方差大
训练不稳定
样本效率较低

3. Actor-Critic：演员和评论家

Actor-Critic 是对 REINFORCE 的改进。
REINFORCE 只靠整局回报 (G_t) ，方差很大。于是 Actor-Critic 加了一个 **Critic** 来估计价值，帮助 Actor 更稳定地更新。
名字很形象：

角色	作用	
Actor	演员，负责选择动作，也就是策略 $(\pi_{\theta}(a$	$s))$
Critic	评论家，负责评价动作好坏，比如估计 $(V(s))$ 或 $(Q(s,a))$	

Actor 做动作，Critic 在旁边点评：

Actor：我在这个状态选择向右。
Critic：这个动作比我预期的好，你以后可以多选。
Critic：这个动作比我预期的差，你以后少选。

Actor-Critic 通常用 **TD error** 来评价动作好坏：

$$\delta = R_{t+1}+\gamma V(S_{t+1})-V(S_t)$$

其中 (δ) 表示：

实际表现 - 预期表现

如果：

$\delta > 0$ ：说明比预期好，增加该动作概率
 $\delta < 0$ ：说明比预期差，降低该动作概率

Actor-Critic 的流程：

1. Actor 根据策略 $\pi_\theta(a|s)$ 选择动作 a
2. 环境返回奖励 r 和下一个状态 s'
3. Critic 计算 TD error:
$$\delta = r + \gamma V(s') - V(s)$$
4. Critic 更新自己的价值函数 V
5. Actor 根据 δ 更新策略:
如果 δ 大, 增加该动作概率
如果 δ 小, 降低该动作概率
6. 进入下一个状态, 继续学习

它和 REINFORCE 的关键区别:

对比	REINFORCE	Actor-Critic
更新依据	整条 episode 的回报 (G_t)	Critic 给出的 TD error
是否必须等 episode 结束	是	不一定
方差	大	较小
是否有价值函数	通常没有	有 Critic
稳定性	较差	更好

一句话:

REINFORCE 是“赛后复盘”, Actor-Critic 是“边演边被点评”。

4. TRPO：限制策略更新步子别太大

TRPO 全称是:

Trust Region Policy Optimization

中文一般叫:

信赖域策略优化

它解决的问题是:

策略梯度更新时, 如果一步更新太大, 策略可能直接崩掉。

比如原策略是:

向左: 0.25
向右: 0.25
向上: 0.25
向下: 0.25

一次更新后突然变成:

向左: 0.99
向右: 0.00
向上: 0.00
向下: 0.01

这就太猛了。强化学习里这种“大跳步”很危险, 可能让策略性能断崖式下降。

TRPO 的思想是:

可以更新策略, 但新策略不能离旧策略太远。

它用 **KL 散度** 衡量新旧策略差距:

$KL(\text{旧策略} || \text{新策略})$

TRPO 的核心约束可以理解为:

$$\max_{\theta} L(\theta) \text{ s.t. } \mathbb{E}_{\pi_{\text{old}}} [KL(\pi_{\text{old}} || \pi_{\theta})] \leq \delta$$

意思是:

目标: 让策略变好
约束: 但新策略和旧策略的差距不能超过 δ

TRPO 的优点:

训练更稳定
避免策略突然崩坏
理论上更有保证

缺点:

实现复杂
计算量大
需要近似二阶优化
工程上不太方便

一句话:

TRPO 像给策略更新套了安全绳, 能往前跳, 但不能一脚蹦出悬崖。

5. PPO：TRPO 的实用简化版

PPO 全称是:

Proximal Policy Optimization

中文一般叫:

近端策略优化

PPO 的目标和 TRPO 类似:

限制策略更新幅度, 让训练更稳定。

但 PPO 不像 TRPO 那样显式求复杂的 KL 约束, 而是用一个非常实用的技巧:

clip 截断

PPO 会计算新旧策略概率比:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

这个比例表示:

新策略选择这个动作的概率
相对于
旧策略选择这个动作的概率
变化了多少

比如：

r = 1.0：没变化
r = 1.2：新策略更倾向这个动作
r = 0.8：新策略更不倾向这个动作

PPO 会把这个比例限制在一个范围内，例如：

[1 - ε, 1 + ε]

常见是：

[0.8, 1.2]

PPO 的核心目标：
$$L^{(CLIP)}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

不用硬背细节，考试理解成：

如果新策略变化太大：
clip 会把它截住
防止策略更新过猛

PPO 的优点：

实现比 TRPO 简单
训练稳定
效果好
工程上非常常用

缺点：

仍然是 on-policy，样本效率不算特别高
超参数也需要调

一句话：

PPO 是 TRPO 的轻装版，把复杂安全绳换成了一个好用的限速器。

6. on-policy 是什么意思？

你表里很多算法写的是 **on-policy**。

on-policy 的意思是：

用当前策略采样的数据，来更新当前策略本身。

例如 PPO：

用 π_{old} 跑环境，收集数据
用这些数据更新策略
更新几轮后，这批数据就不能一直用了
因为策略已经变了

所以 on-policy 方法通常：

训练比较稳定
但是样本利用率较低

off-policy 则是：

可以用旧策略、其他策略、经验回放中的数据来学习当前策略。

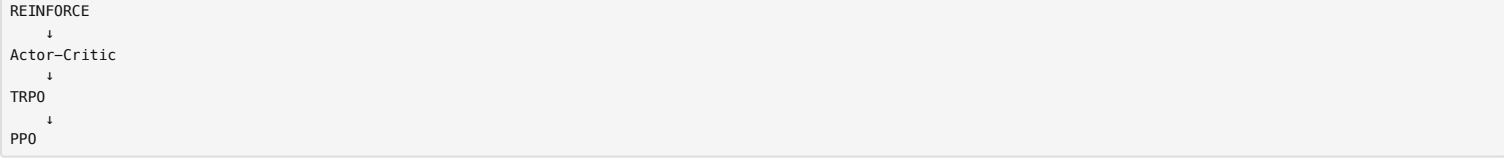
比如：

Q-learning
DQN
DDPG
SAC

通常更省样本，但训练可能更复杂。

7. 这几个算法的关系

可以按进化路线理解：



它们大概是在解决同一个问题的不同阶段：

算法	解决的问题
REINFORCE	最基础的策略梯度，用真实回报更新策略
Actor-Critic	REINFORCE 方差太大，所以加 Critic 评价
TRPO	策略更新太猛容易崩，所以加 KL 约束
PPO	TRPO 太复杂，所以用 clip 简化

8. 和 Value-based 方法的区别

对比	Value-based	Policy-based	
学习对象	$V(s)$ 或 $Q(s,a)$	策略 $\pi(a$	$s)$
动作选择	根据价值选最大动作	直接由策略输出动作概率	
适合动作空间	离散动作更常见	离散和连续动作都适合	
是否能学随机策略	不太自然	很自然	
代表算法	Q-learning、SARSA、DQN	REINFORCE、A2C/A3C、TRPO、PPO	

9. 考试怎么写？

如果题目问：

简述 REINFORCE、Actor-Critic、TRPO、PPO。

可以这样答：
REINFORCE 是一种蒙特卡洛策略梯度算法，不需要环境模型，通常属于 on-policy 方法。它通过当前策略采样完整 episode，在 episode 结束后计算真实回报 (G_t)，并利用回报调整策略参数，使高回报动作的选择概率增大。其缺点是必须等 episode 结束后更新，且方差较大。
Actor-Critic 方法同时包含 Actor 和 Critic 两部分。Actor 表示策略，负责根据状态选择动作；Critic 表示价值函数，负责评价当前动作或状态的好坏。Actor 根据 Critic 给出的 TD error 或优势函数更新策略，Critic 则根据采样经验更新价值函数。相比 REINFORCE，Actor-Critic 不一定需要等待完整 episode，方差更小，学习更稳定。
TRPO 是一种 on-policy 策略优化算法，其核心思想是在更新策略时限制新旧策略之间的 KL 散度，使策略更新保持在一个信赖区域内，避免更新过大导致性能崩溃。TRPO 稳定性较好，但实现和计算较复杂。
PPO 是对 TRPO 的简化和实用化改进。它通过 clip 截断新旧策略概率比，限制策略更新幅度，从而在保证稳定性的同时降低实现复杂度。PPO 通常属于 on-policy 算法，是目前使用非常广泛的策略梯度方法。

10. 最短背诵版

REINFORCE： 蒙特卡洛策略梯度。 等一整局结束，用真实回报 G 更新策略。
Actor-Critic： Actor 选动作，Critic 评价动作。 用 Critic 降低策略梯度方差。
TRPO： 用 KL 约束限制新旧策略差距。 防止策略更新过大。
PPO： 用 clip 限制新旧策略概率比。 是 TRPO 的简单实用版本。

再压缩成一句：

REINFORCE 直接用回报改策略；Actor-Critic 加评论家降低方差；TRPO 用 KL 保守更新；PPO 用 clip 简化 TRPO。

下面我把你图里的 三次作业 分开写成可以直接交作业的版本，并在每题后面加详细解释。整体围绕强化学习第五章：时序差分 TD、SARSA、Q-learning、Dyna-Q，再加第一题里的矩阵计算与收敛性分析。

作业一：矩阵 (A)、向量 (x)、特征值与收敛性分析

题中给出：

```
[
A=
\begin{pmatrix}
0&1&1&0&0\\
0&0&1&1&0\\
0&0&0&1&1\\
0&0&0&0&0\\
0&0&0&0&0
\end{pmatrix}
]
要求：
[
x=(I-A)^{-1}b
]
并求：
[
x=(x_5,x_4,x_3,x_2,x_1)^T
]
---
```

1. 先写出 (I-A)

```
[
I-A=
\begin{pmatrix}
1&-1&-1&0&0\\
0&1&-1&-1&0\\
0&0&1&-1&-1\\
0&0&0&1&0\\
0&0&0&0&1
\end{pmatrix}
]
因为 (A) 是严格上三角矩阵，所以它是一个非常“乖”的矩阵，像一条只能往右上方流动的小溪，没有回路。
---
```

2. 计算 ((I-A)^{-1})

由于 (A) 是严格上三角矩阵，并且满足：

```
[
A^4=0
]
所以：
[
(I-A)^{-1}=I+A+A^2+A^3
]
计算得到：
[
(I-A)^{-1}
=====
\begin{pmatrix}
1&1&2&3&2\\
0&1&1&2&1\\
0&0&1&1&1\\
0&0&0&1&0\\
0&0&0&0&1
\end{pmatrix}
]
所以如果：
[
```

```
b=(b_5,b_4,b_3,b_2,b_1)^T
]
那么：
[
x=(I-A)^{-1}b
]
即：
[
\begin{pmatrix}
x_5\
x_4\
x_3\
x_2\
x_1
\end{pmatrix}
=====
\begin{pmatrix}
1&1&2&3&2\
0&1&1&2&1\
0&0&1&1&1\
0&0&0&1&0\
0&0&0&0&1
\end{pmatrix}
\begin{pmatrix}
b_5\
b_4\
b_3\
b_2\
b_1
\end{pmatrix}
]
因此：
[
x_5=b_5+b_4+2b_3+3b_2+2b_1
]
[
x_4=b_4+b_3+2b_2+b_1
]
[
x_3=b_3+b_2+b_1
]
[
x_2=b_2
]
[
x_1=b_1
]
---
```

3. 如果题目默认 (b=(0,0,0,1,1)^T)

很多这类题中，最后两个状态是终止状态，常设：

```
[
b=(0,0,0,1,1)^T
]
那么：
[
x=
\begin{pmatrix}
1&1&2&3&2\
0&1&1&2&1\
0&0&1&1&1\
0&0&0&1&0\
0&0&0&0&1
\end{pmatrix}
\begin{pmatrix}
0\
0\
0\
1\
1
\end{pmatrix}
=====
\begin{pmatrix}
5\
3\
2\
1\
1
\end{pmatrix}
]
所以：
[
x=(x_5,x_4,x_3,x_2,x_1)^T=(5,3,2,1,1)^T
]
也就是：
[
x_5=5,\quad x_4=3,\quad x_3=2,\quad x_2=1,\quad x_1=1
]
这个结果像斐波那契数列的小尾巴：终止状态给 (1)，往前的状态不断把后继状态的值加起来。
---
```

4. 特征值与收敛性分析

矩阵 (A) 是严格上三角矩阵，它的主对角线全是 (0)。
对于上三角矩阵，特征值就是主对角线元素。

所以：

```
[
\lambda_1=\lambda_2=\lambda_3=\lambda_4=\lambda_5=0
]
```

因此谱半径为：

```
[
\rho(A)=\max_i|\lambda_i|=0
]
```

因为：

```
[
\rho(A)=0<1
]
```

所以迭代：

```
[
x_{k+1}=Ax_k+b
]
```

一定收敛。

并且由于：

```
[
A^4=0
]
```

所以从 (x_0=0) 开始，最多迭代 (4) 次就可以达到精确解：

```
[
x^*=b+Ab+A^2b+A^3b
]
```

因此本题中的矩阵迭代不仅收敛，而且是有限步收敛。

作业二：TD、SARSA、多步 SARSA、Q-learning 公式与比较

1. 时序差分 TD 的核心更新公式

TD 方法的核心思想是：

不等一整条 episode 结束，而是每走一步就利用当前奖励和下一个状态的估计值进行更新。

最经典的是 TD(0)：

```
[
V(S_t)\leftarrow V(S_t)+\alpha\left[R_{t+1}+\gamma V(S_{t+1})-V(S_t)\right]
]
```

其中：

```
[
R_{t+1}+\gamma V(S_{t+1})
]
```

叫做 TD 目标。

```
[
R_{t+1}+\gamma V(S_{t+1})-V(S_t)
]
```

叫做 TD 误差，记为：

```
[
\delta_t=R_{t+1}+\gamma V(S_{t+1})-V(S_t)
]
```

所以 TD 更新也可以写成：

```
[
V(S_t)\leftarrow V(S_t)+\alpha\delta_t
]
```

2. SARSA 的核心更新公式

SARSA 是一种 **on-policy** 的动作价值学习算法。

它名字来自五元组：

```
[
S_t,A_t,R_{t+1},S_{t+1},A_{t+1}
]
```

也就是：

当前状态 S
当前动作 A
奖励 R
下一个状态 S'
下一个动作 A'

SARSA 更新公式为：

```
[
Q(S_t,A_t)\leftarrow Q(S_t,A_t)+\alpha\left[R_{t+1}+\gamma Q(S_{t+1},A_{t+1})-Q(S_t,A_t)\right]
]
```

注意，SARSA 使用的下一个动作 (A_{t+1}) 是智能体按照当前策略真实选出来的动作。

也就是说，如果当前策略是 (epsilon)-greedy，那么 SARSA 的更新也会考虑探索动作。

3. 多步 SARSA 的核心更新公式

普通 SARSA 只看一步：

```
[
R_{t+1}+\gamma Q(S_{t+1},A_{t+1})
]
```

多步 SARSA 会看未来 (n) 步奖励。

定义 (n) 步回报：

```
[
G_t^{\{n\}}
=====
R_{t+1}
+\gamma R_{t+2}
+\cdots
+\gamma^{n-1}R_{t+n}
+\gamma^n Q(S_{t+n},A_{t+n})
]
```

然后更新：

[
Q(S_t,A_t)\leftarrow Q(S_t,A_t)+\alpha\left[G_t^{(n)}-Q(S_t,A_t)\right]
]

它介于一步 TD 和 MC 之间：

方法	看多远
一步 SARSA	看一步
多步 SARSA	看 (n) 步
MC	看到 episode 结束

所以多步 SARSA 像是折中方案：既不像 TD 那样只盯脚下一步，也不像 MC 那样非要等整场电影放完。

4. Q-learning 的核心更新公式

Q-learning 是一种 **off-policy** 的动作价值学习算法。
它的更新公式为：

[
Q(S_t,A_t)\leftarrow Q(S_t,A_t)+\alpha\left[R_{t+1}+\gamma\max_a Q(S_{t+1},a)-Q(S_t,A_t)\right]
]

重点在：

[
\max_a Q(S_{t+1},a)
]

也就是说，Q-learning 不管下一步真实执行了什么动作，它更新时都假设下一步会选择最优动作。
所以它学习的是最优策略对应的 (Q^*)。

作业二第 2 问：SARSA 与 Q-learning 的迭代过程有什么不同？

核心区别一句话：

SARSA 用真实选择的下一个动作更新，是 on-policy；Q-learning 用下一个状态的最大 Q 值更新，是 off-policy。

1. SARSA 的迭代过程

SARSA 的流程是：

```
初始化 Q(s,a)

每个 episode:
    初始化状态 S
    根据当前策略选择动作 A

    while S 不是终止状态:
        执行动作 A
        得到奖励 R 和下一个状态 S'

        根据当前策略在 S' 中选择下一个动作 A'

        用公式更新:
            Q(S,A) ← Q(S,A) + α[R + γQ(S',A') - Q(S,A)]

        S ← S'
        A ← A'
```

注意：SARSA 更新时用的是 (Q(S',A'))，其中 (A') 是当前策略真实选出来的动作。
如果当前策略有探索，比如 (epsilon)-greedy，那么 (A') 可能不是最优动作。
因此 SARSA 会把探索风险也考虑进去。

2. Q-learning 的迭代过程

Q-learning 的流程是：

```
初始化 Q(s,a)

每个 episode:
    初始化状态 S

    while S 不是终止状态:
        根据行为策略选择动作 A
        执行动作 A
        得到奖励 R 和下一个状态 S'

        用公式更新:
            Q(S,A) ← Q(S,A) + α[R + γ max_a Q(S',a) - Q(S,A)]

        S ← S'
```

注意：Q-learning 更新时用的是：

[
\max_a Q(S',a)
]

它不管下一步真实选了什么动作，只看下一个状态里理论上的最优动作。
因此 Q-learning 学的是目标策略，也就是最优贪心策略。

3. 对比总结

对比点	SARSA	Q-learning
策略类型	on-policy	off-policy
更新目标	(R+\gamma Q(S',A'))	(R+\gamma\max_a Q(S',a))
是否考虑真实探索动作	考虑	不考虑
学到的策略	当前行为策略对应的价值	最优贪心策略对应的价值
风格	更保守	更激进
安全性	通常更安全	可能更冒险
代表例子	Cliff Walking 中绕远路	Cliff Walking 中贴悬崖走

在悬崖行走实验中，SARSA 知道自己有 (epsilon) 的概率会随机探索，所以会倾向于远离悬崖；Q-learning 只看最优动作，容易学出贴着悬崖走的最短路径，但实际探索时可能掉下去。

作业二第 3 问：结合本周实验，说明上述方法的优缺点

本周实验内容是第五章时序差分算法，主要涉及 TD、SARSA、多步 SARSA、Q-learning 等方法。可以这样写：

1. TD 方法优缺点

TD 方法的优点是：

第一，它不需要环境模型，不需要知道状态转移概率 ($P(s'|s,a)$)。

第二，它不需要等 episode 结束，每走一步就可以更新。

第三，相比蒙特卡洛方法，TD 的学习速度通常更快。

缺点是：

TD 使用的是估计值 ($V(S_{t+1})$)，不是完整真实回报，所以存在一定偏差。如果初始价值估计不准，早期学习结果可能会受到影响。

2. SARSA 方法优缺点

SARSA 的优点是：

它是 on-policy 方法，更新时考虑了智能体实际会执行的动作，因此在带探索的情况下更稳健、更保守。在一些有危险区域的环境中，例如 Cliff Walking，SARSA 往往能学到更安全的路线。

缺点是：

因为 SARSA 会考虑探索动作带来的风险，所以有时学到的路径不是理论上的最短路径，收敛速度可能比 Q-learning 慢一些。

3. 多步 SARSA 方法优缺点

多步 SARSA 的优点是：

它结合了一步 TD 和蒙特卡洛方法的特点。相比一步 SARSA，它能利用更多未来奖励信息；相比 MC，它又不必等到完整 episode 结束。

缺点是：

多步长度 (n) 需要选择合适。如果 (n) 太小，信息传播较慢；如果 (n) 太大，回报方差会增大，学习可能不稳定。

4. Q-learning 方法优缺点

Q-learning 的优点是：

它是 off-policy 方法，可以在探索的同时学习最优贪心策略。它更新时直接使用下一个状态的最大动作价值：

```
[
\max_a Q(S',a)
]
```

因此目标明确，通常收敛较快，最终容易学到最优路径。

缺点是：

它不考虑实际探索动作的风险。在一些危险环境中，Q-learning 可能学习到贴近危险区域的最短路径，理论上最优，但实际执行时由于探索动作存在，可能导致较高失败率。

5. 实验结果可以这样总结

在本周实验中，可以观察到：TD 类算法都能在不需要环境模型的情况下，通过与环境交互逐渐学习状态价值或动作价值。SARSA 因为使用实际执行的下一个动作更新，表现更保守，路径更安全；Q-learning 因为使用最大动作价值更新，学习更偏向最优路径，但在探索阶段可能风险更高；多步 SARSA 通过引入多步回报，加快了奖励信息向前传播，但需要合理选择步数 (n)。

可以作为作业结论：

TD 方法相比 MC 方法更新更及时，不需要等待完整 episode；SARSA 更安全但可能保守；Q-learning 更容易学到最优路径但可能忽视探索风险；多步 SARSA 在一步 TD 和 MC 之间折中，能加快学习但对步数参数较敏感。

作业三：Dyna-Q 解析

图中说：

Dyna-Q 本质上就是 Q-learning，但是在经验数据的使用上与传统 Q-learning 有较大区别。

这句话是对的。下面展开成完整答案。

1. 普通 Q-learning 的学习方式

普通 Q-learning 每次和真实环境交互一次，得到一条真实经验：

```
[
(s,a,r,s')
]
```

然后只做一次更新：

```
[
Q(s,a)←Q(s,a)+α[r+γmax_{a'}Q(s',a')-Q(s,a)]
]
```

也就是说：

真实走一步
更新一次 Q 值
然后这条经验基本就用完了

所以传统 Q-learning 的样本利用率不高。

2. Dyna-Q 的核心思想

Dyna-Q 在 Q-learning 的基础上加入了一个环境模型：

```
[
\hat{M}(s,a)→(r,s')
]
```

这个模型记录智能体已经经历过的状态转移。

每次真实交互后，Dyna-Q 会做三件事：

1. 用真实经验更新 Q 值
2. 用真实经验更新环境模型 M
3. 从模型中采样旧的状态动作对，生成模拟经验，再更新 Q 值

所以 Dyna-Q 的学习不是：

真实经验用一次就丢

而是：

真实经验存入模型，以后可以反复模拟使用

这就是 Dyna-Q 样本效率高的原因。

3. Dyna-Q 的算法流程

可以写成伪代码：

```
初始化 Q(s,a)
初始化模型 Model(s,a)

每个 episode:
    初始化状态 s

    while s 不是终止状态:

        1. 根据 ε-greedy 策略选择动作 a

        2. 在真实环境中执行动作 a
           得到奖励 r 和下一个状态 s'

        3. 直接强化学习更新:
            $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ 

        4. 模型学习:
            $Model(s,a) \leftarrow (r,s')$ 

        5. 规划学习:
           重复 n 次:
               随机选择一个以前访问过的状态动作对 (s_m,a_m)

               根据模型得到:
                   (r_m,s'_m) = Model(s_m,a_m)

               用模拟经验更新:
                    $Q(s_m,a_m) \leftarrow Q(s_m,a_m) + \alpha[r_m + \gamma \max_{a'} Q(s'_m,a') - Q(s_m,a_m)]$ 

        6.  $s \leftarrow s'$ 
```

4. Dyna-Q 为什么比 Q-learning 快?

普通 Q-learning:

一次真实交互 = 一次 Q 更新

Dyna-Q:

一次真实交互 = 一次真实 Q 更新 + n 次模拟 Q 更新

也就是说，如果规划步数是 (n)，那么一次真实经验被放大为：

[
n+1
]

次学习更新。

所以图中说：

一次真实交互被“放大”为 (n+1) 次学习更新。

这个理解非常关键。

5. Dyna-Q 和 Q-learning 的区别

对比点	Q-learning	Dyna-Q
是否学习环境模型	否	是
是否使用模拟经验	否	是
每次真实交互后的更新次数	1 次	(1+n) 次
样本利用率	较低	较高
收敛速度	较慢	通常更快
对模型依赖	无	依赖模型准确性

6. Dyna-Q 的优点

第一，样本利用率高。

因为每条真实经验都会进入模型，之后可以反复被采样使用。

第二，学习速度快。

模型生成的模拟经验能让价值信息更快传播。

第三，结合了 model-free 和 model-based 的思想。

真实经验更新 Q 值属于 model-free，利用模型进行模拟规划属于 model-based。

可以说：

Dyna-Q 是 Q-learning 加了一个“脑内演练器”。

7. Dyna-Q 的缺点

第一，如果模型错误，模拟经验也会错误。

例如模型错误地认为某个动作会到达高奖励状态，那么智能体会反复用错误经验训练，导致 Q 值偏移。

第二，在环境发生变化时，旧模型可能过时。

如果环境中的障碍物或奖励发生变化，Dyna-Q 仍可能根据旧模型做规划，导致学习滞后。

第三，规划步数 (n) 过大时，计算量会增加。

虽然样本效率提高，但每次真实交互后的计算成本也更高。

总结版：三次作业可以背这一段

第一题中，矩阵 (A) 是严格上三角矩阵，因此所有特征值均为 (0)，谱半径 ($\rho(A)=0<1$)，所以迭代 ($x_{k+1}=Ax_k+b$) 收敛。由于 ($A^4=0$)，有 ($(I-A)^{-1}=I+A+A^2+A^3$)。若 ($b=(0,0,0,1,1)^T$)，则 ($x=(5,3,2,1,1)^T$)。

第二题中，TD 方法通过一步奖励和下一状态估计值更新价值函数；SARSA 使用真实选择的下一动作 (A') 更新，是 on-policy；多步 SARSA 使用 (n) 步回报更新，介于一步 TD 和 MC 之间；Q-learning 使用 ($\max_{a'} Q(S',a)$) 更新，是 off-policy。SARSA 更保守安全，Q-learning 更偏向最优路径但可能忽视探索风险。

第三题中，Dyna-Q 是在 Q-learning 基础上加入环境模型和规划步骤的算法。每次真实交互后，它不仅用真实经验更新 Q 值，还用该经验更新模型，并利用模型生成 (n) 次模拟经验继续更新 Q 值。因此 Dyna-Q 相比普通 Q-learning 样本利用率更高，收敛速度更快，但缺点是依赖模型准确性，且规划步数过大时计算量增加。